SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 3

Name of Student : Mandala Naveen

Enrollment No. : 2303A51436

Batch No. 21

Question 1: Zero-Shot Prompting (Palindrome Number Program)
Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.
Task:
- **Record the AI-generated code.**
- **Test the code with multiple inputs.**
- **Identify any logical errors or missing edge-case handling.**

Code:-

Assignment3.py > ...

```python
#Task1
from logging import root


def palindrome(number):
    str_num = str(number)
    return str_num == str_num[::-1]
print(palindrome(121))
print(palindrome(123))
print(palindrome(12321))
print(palindrome(45654))
print(palindrome(789))
```

Output:-

```
True
False
True
True
False
```

Question 2: One-Shot Prompting (Factorial Calculation)
Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.
Example:
Input: 5 → Output: 120
Task:
• Compare the generated code with a zero-shot solution.
• Examine improvements in clarity and correctness.

Prompt:-
input: 5 => output: 120 write a function to calculate factorial of a number

Code:-

Assignment3.py > ...

```python
#Task2
# input: 5 -> output: 120 write a function to calculate factorial of a number
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
print(factorial(5))
print(factorial(0))
print(factorial(6))
```

Output:-

```
120
1
720
```

Question 3: Few-Shot Prompting (Armstrong Number Check)
Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.
Examples:
• Input: 153 → Output: Armstrong Number
• Input: 370 → Output: Armstrong Number
• Input: 123 → Output: Not an Armstrong Number
Task:
• Analyze how multiple examples influence code structure and accuracy.
• Test the function with boundary values and invalid inputs.
(Optional Extension)

Prompt:-
input: 370 => output: Armstrong write a function to check if a number is an Armstrong number.

Code:-

Assignment3.py > CheckArmstrong

```python
#Task3
# input: 153 -> output: Armstrong
# input: 123 -> output: Not Armstrong
# input: 370 -> output: Armstrong
# write a program check whether function to check if a number is an Armstrong number or not Armstrong number.
def is_Armstrong(number):
    num_str = str(number)
    num_digits = len(num_str)
    sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
    return sum_of_powers == number
def CheckArmstrong(number):
    if is_Armstrong(number):
        print("Armstrong")
    else:
        print("Not Armstrong")
CheckArmstrong(153)
CheckArmstrong(123)
CheckArmstrong(370)
CheckArmstrong(9474)
```

Output:-

```
Armstrong
Not Armstrong
Armstrong
Armstrong
```

Question 4: Context-Managed Prompting (Optimized Number Classification)
Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.
Task:
- **Ensure proper input validation.**
- **Optimize the logic for efficiency.**
- **Compare the output with earlier prompting strategies.**

Prompt:-
write a program on a context-managed that classifies number as prime, composite or neither.

Code:-

Assignment3.py > ...

```python
#Task4
# write a program on a context-managed that classifies number as prime, composite or neither.
class NumberClassifier:
    def __init__(self, number):
        self.number = number

    def __enter__(self):
        if self.number <= 1:
            self.classification = "Neither prime nor composite"
        elif self.number == 2:
            self.classification = "Prime"
        else:
            for i in range(2, int(self.number ** 0.5) + 1):
                if self.number % i == 0:
                    self.classification = "Composite"
                    break
            else:
                self.classification = "Prime"
        return self.classification

    def __exit__(self, exc_type, exc_value, traceback):
        pass
with NumberClassifier(7) as classification:
    print(classification)
with NumberClassifier(10) as classification:
    print(classification)
with NumberClassifier(1) as classification:
    print(classification)
with NumberClassifier(13) as classification:
    print(classification)
with NumberClassifier(15) as classification:
    print(classification)
with NumberClassifier(0) as classification:
    print(classification)
```

Output:-

```
Prime
Composite
Neither prime nor composite
Prime
Composite
Neither prime nor composite
```

Question 5: Zero-Shot Prompting (Perfect Number Check)
Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.
Task:
• Record the AI-generated code.
• Test the program with multiple inputs.

• Identify any missing conditions or inefficiencies in the logic.

Code:-

```
#Task5
def perfect_number(n):
    if n < 2:
        return False
    divisors_sum = sum(i for i in range(1, n) if n % i == 0)
    return divisors_sum == n
print(perfect_number(6))
print(perfect_number(28))
print(perfect_number(12))
print(perfect_number(496))
print(perfect_number(15))
```

Output:-

```
True
True
False
True
False
```

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)
Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.
Examples:
• Input: 8 → Output: Even
• Input: 15 → Output: Odd
• Input: 0 → Output: Even
Task:
• Analyze how examples improve input handling and output clarity.
• Test the program with negative numbers and non-integer inputs.

Prompt:-
input: 8 -> output: Even

input: 15 -> output: Odd
input: 0 -> output: Even
write a function to check if a number is even or odd.

Code:-

```
Assignment3.py > ...
#Task6
# input: 8 -> output: Even
# input: 15 -> output: Odd
# input: 0 -> output: Even write a function to check if a number is even or odd.
def even_or_odd(number):
    return "Even" if number % 2 == 0 else "Odd"
print(even_or_odd(8))
print(even_or_odd(6.2))
print(even_or_odd(6/3))
print(even_or_odd(5**0.5))
print(even_or_odd(3.14159))
```

Output:-

```
Even
Odd
Even
Odd
Odd
```